RAPPORT PROJET M1

RAPPORT EFFECTUÉ DANS LE CADRE DU PROJET DE SEMESTRE 8

Nicolas CAMINADE

Tanguy DECABRAT

Anya BAZIZI

Younes OUAAMMOU

Thomas-Alexandre MOREAU



Sommaire

1 Présentation du projet **2**
- 1.1 Historique des Amazones . . . 2
- 1.2 Règles . . . 2
- 1.3 Complexité . . . 3

2 Structures et algorithmes **3**
- 2.1 Structures de données . . . 3
  - 2.1.1 Réseaux . . . 3
  - 2.1.2 IA . . . 4
  - 2.1.3 Moteur de Jeu . . . 4
  - 2.1.4 Interface Graphique (GUI) . . . 5
  - 2.1.5 Intégration et CLI . . . 6
- 2.2 Algorithmes utilisés . . . 7
  - 2.2.1 Réseaux . . . 7
  - 2.2.2 IA . . . 7
  - 2.2.3 Moteur de jeu . . . 8
  - 2.2.4 CLI . . . 8

3 Architecture **9**
- 3.1 Modules . . . 9
  - 3.1.1 IA . . . 9
  - 3.1.2 Réseaux . . . 9
- 3.2 Dépendances . . . 10

4 Spécifications **11**
- 4.1 Réseaux . . . 11
  - 4.1.1 Découverte de service . . . 11

5 Transparence IA **11**

Références bibliographiques **12**



Présentation du projet

1.1 Historique des Amazones

Le jeu des Amazones, créé en 1988 par l'Argentin Walter Zamkauskas, est un défi majeur pour l'informatique, dont l'histoire est la suivante :

- En 2005, l'équipe de Furtak prouve mathématiquement que résoudre ce jeu est PSPACE-complet, une classe de difficulté bien supérieure à celle des problèmes NP standards. [4]
- Richard Lorentz, avec son programme Invader, démontre ensuite que l'approche statistique du Monte Carlo Tree Search (MCTS) est la seule viable face à l'explosion combinatoire du jeu en 2017.[7]
- Depuis lors, des chercheurs intègrent désormais le Deep Learning et les réseaux de neurones pour tenter de surpasser cet algorithme.[9]

1.2 Règles

Le Jeu des Amazones se pratique sur un damier, traditionnellement de 10×10 cases. Chaque joueur possède quatre amazones, placées dans une configuration initiale spécifique qui définit le point de départ .

Objectif du jeu : Le but est de complètement bloquer les mouvements de l'adversaire en occupant ou en bloquant stratégiquement l'espace. La victoire est acquise lorsqu'un joueur parvient à bloquer toutes les amazones adverses, ne lui laissant plus aucune possibilité de mouvement.

Déroulement d'un tour : Chaque action d'un joueur est impérativement composée de deux phases successives :

1. **Le déplacement :** Le joueur choisit l'une de ses amazones et la déplace en ligne droite (verticalement, horizontalement ou en diagonale) vers une case libre. Le mouvement est identique à celui de la dame aux échecs, mais l'amazone ne peut ni traverser ni s'arrêter sur une case occupée par une autre pièce ou une flèche.
2. **Le tir de flèche :** Après son déplacement, cette même amazone doit tirer une flèche depuis sa nouvelle position. La flèche suit les mêmes règles de trajectoire qu'une amazone. La case d'arrivée de la flèche est alors marquée d'un blocage définitif pour le reste de la partie.

Fin de partie et variantes Contrairement à d'autres jeux de plateau, le Jeu des Amazones ne connaît pas de situation de match nul par répétition ou par épuisement de matériel. Puisque chaque tour réduit irrémédiablement le nombre de cases disponibles, le jeu se termine forcément par le blocage complet d'un joueur. Bien que le format 10×10 soit le standard, il existe des variantes (6×6 ou 8×8) avec un nombre d'amazones adapté, permettant des parties plus rapides tout en conservant la complexité tactique du jeu.

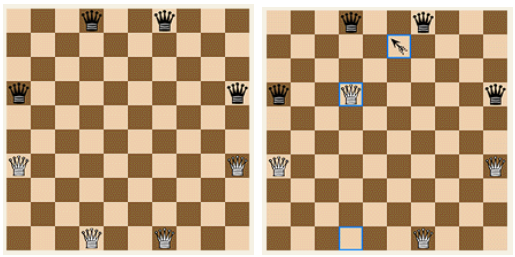


FIGURE 1 – Évolution d'une partie : position initiale et premier coup [12] [13]

1.3 Complexité

Le jeu des Amazones est un jeu d'une très grande complexité [2].

	Échecs	**Amazones**	**Go**
Facteur Branch. Initial	~ 20	~ 2176	~ 361
Facteur Branch. Moyen	~ 35	~ 374 joueur de départ ~ 299 deuxième joueur ~ 50 durant la fin de partie	~ 250
Nombre de tours moyen	~ 80	~ 84	~ 150
Cmplx. Arbre de jeu	$\sim 10^{123}$	$\sim 1.09 * 10^{212}$	$\sim 10^{360}$
Cmplx. Espaces d'états	de $\sim 3.7 * 10^{43}$ à $\sim 10^{52}$	$\sim 6.45* 10^{40}$	$\sim 10^{170}$

TABLE 1 – Comparaison des complexités [1] [6]

Comme l'illustre le tableau comparatif, la complexité des Amazones réside dans son facteur de branchement initial extrême, surpassant largement celui des échecs et du Go. Bien que la profondeur de jeu soit modérée, cette "largeur" excessive génère un arbre de jeu colossal (10^{212}), empêchant une exploration exhaustive de l'arbre en raison de l'explosion combinatoire des possibilités de jeu. Pour ces raisons, algorithmiquement, le défi se rapproche ainsi davantage de la résolution du Go que des échecs.

Pour contourner cette explosion combinatoire, les approches modernes s'appuient sur la sélectivité : soit par des solveurs de preuve comme *df-pn* [8], soit par des algorithmes de jeu utilisant du *Forward Pruning* [3] ou d'autres méthodes statistiques que nous mettons en place dans nos IA, comme dans *MCTS* via l'*Upper Confidence bounds applied to Trees* (UCT) [5] ou encore le *Machine Learning* (ML).

Structures et algorithmes

2.1 Structures de données

2.1.1 Réseaux

Pour ce qui est de la partie réseaux du projet, on utilise plusieurs structures afin de pouvoir interagir adéquatement entre les clients et le serveur.



— Au niveau du serveur, on utilise un tableau de structures nommé "Connected-Client". Ce choix a été fait par souci de simplicité et de performance. Dans ce tableau, chaque client possède un état "state" permettant d'appliquer le bon comportement au programme dans chacun des cas où il faut vérifier les états des clients ;
— Il est possible d'avoir plusieurs clients pour un même serveur. De ce fait, nous disposons d'un buffer TCP permettant de gérer la fragmentation des packets qui pourraient ne pas arriver dans de bonnes conditions ou dans le bon ordre. Par exemple, nous pourrions avoir le cas où un message arriverait en 2 morceaux. Ce tableau possède aussi, pour chaque client, leurs informations telles que leurs statistiques, noms et scores.
— En complément de la gestion TCP, nous avons intégré une structure spécifique nommée DiscoveryContext. Celle-ci repose sur le protocole UDP en mode Broadcast. Elle agit comme une balise autonome, elle encapsule un socket UDP dédié et un timer (last broadcast time). Son rôle est de diffuser périodiquement un ping du serveur sur le réseau local sans bloquer la boucle principale. Cette architecture découplée permet aux clients d'utiliser la commande server list pour détecter les parties disponibles sans connaître l'adresse IP du serveur à l'avance, tout en isolant strictement la logique "sans connexion" (UDP) de la logique de jeu (TCP).

2.1.2 IA

Nous connaissons déjà certaines IAs pré-requises au sein de notre architecture, et avons souhaité les optimiser au maximum afin de permettre une meilleure expérience de jeu, mais aussi une meilleure qualité des coups joués.

De ce fait, nous avons mis en place plusieurs principes :

— L'utilisation de structures C simulant des interfaces pour réduire les dépendances et permettre le polymorphisme du code ;
— Via une Factory, il est possible de générer une IA spécifique via un tableau de paramètres passé à la méthode de création, pour faciliter l'utilisation des IAs ;
— L'utilisation d'arbres LCRS (Left-Children Right-Sibling) ([10]) afin de réduire la charge sur la mémoire et sur le CPU pour le calcul. Le but est de réduire au plus possible la taille de la structure des millions de nœuds que nous générerons, afin d'alléger la charge CPU/MÉMOIRE et de réduire les coûts par rapport à un tableau de pointeurs classique (possiblement de taille ∼2176, comme vu dans la section 1.3, qui se verrait multiplié par la taille de la structure pour chaque nœud, d'au minimum 3 octets...).

2.1.3 Moteur de Jeu

L'architecture des données repose intégralement sur l'utilisation de Bitboards. Le jeu des Amazones se jouant sur un plateau allant jusqu'à 11×11 cases, une représentation binaire nécessite 121 bits, ce qui excède la capacité des registres standards de 64 bits.

Pour pallier à l'absence du type `uint128_t` dans le standard C11 (et garantir la portabilité sous MSVC), nous avons conçu une structure composite personnalisée. Le plateau est segmenté en deux entiers non-signés de 64 bits :

— `low` : encode les cases d'indices 0 à 63.
— `high` : encode les cases d'indices 64 à 120 (les bits restants étant inutilisés).

La contrainte est de faire agir ces deux entiers comme une seule entité. Nous avons donc encapsulé la logique binaire dans des fonctions dédiées. Si les opérateurs logiques



AND, OR, XOR s'appliquent indépendamment sur chaque moitié, les décalages (SHIFT) nécessitent une propagation : les bits qui "débordent" de la partie basse sont récupérés pour être réinjectés dans la partie haute. Cette mécanique simule un type 128 bits unique, rendant la complexité interne invisible pour le reste du moteur.

Cette approche permet de traiter l'ensemble du plateau en temps constant $\mathcal{O}(1)$ contre une complexité en $\mathcal{O}(N)$ en utilisant un tableau 2D. Cette optimisation permet de maximiser le nombre de nœuds explorés par l'IA.

2.1.4 Interface Graphique (GUI)

Pour l'interface graphique, nous utiliserons la bibliothèque LibGtk. Les versions 4.X offrent énormément de possibilités autant du côté de l'affichage que des interactions utilisateur, en plus d'être relativement intuitives à l'utilisation.

Gtk utilise une application principale (GtkApplication) comme structure, ainsi qu'une fenêtre (GtkWindow) pour représenter la partie visuelle de l'application. On peut ensuite construire différents modules personnalisés (GtkWidget) comme le plateau du jeu par exemple, y ajouter des éléments dynamiques (GtkButton, GtkOverlay, ...), et également intégrer un système d'empilement de pages (GtkStack) pour fluidifier le changement de fenêtre en simulant un changement de fenêtre au lieu de recréer une nouvelle page à chaque fois.

La structure simplifiée de l'interface du jeu serait de cette forme :

- **GtkApplication** Cœur de l'application
  - **GtkWindow** Fenêtre graphique de l'application
    - **GtkOverlay** Module permettant de générer des alertes à l'écran, facilement modifiable à la volée. Donc le placer aussi haut dans la hiérarchie permet de ne pas en créer un par page
      - **GtkStack** Gestionnaire de pages
        - **GtkWidget** Menu Principal
          - **GtkButton**
          - **...**
        - **GtkWidget** Menu Singleplayer
        - **...**

L'interface aura pour but d'être claire et simple d'utilisation, reprenant des codes visuels basiques du jeu vidéo afin de ne pas perdre l'utilisateur. Voici deux schémas afin d'illustrer ces propos :

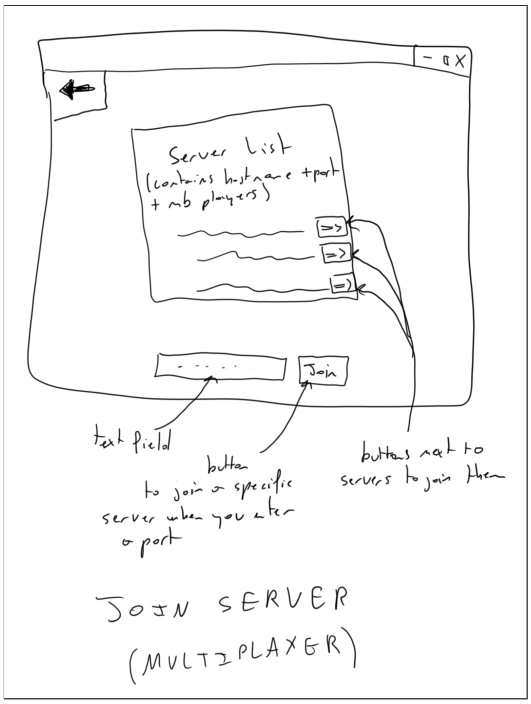


FIGURE 2 – Liste des serveurs

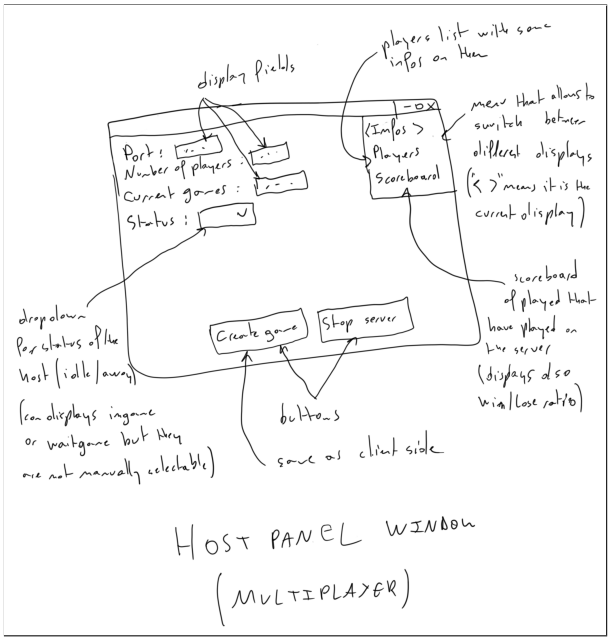


FIGURE 3 – Menu pour le host

Pour construire l'interface avec Gtk, nous passerons par des fichiers XML. Passer par ce langage à balises simplifie la construction de l'interface en plus d'avoir une bonne intégration à Gtk par défaut. Plus précisément, les parties statiques de l'interface seront directement écrites dans les fichiers XML, tandis que les éléments dynamiques tels que les boutons ou les alertes seront codés dans les fichiers C, et feront référence à des tags de certaines balises du XML afin de pouvoir les modifier grâce au code C.

De plus, Gtk offre la possibilité d'intégrer du Drag'n'Drop (pour déplacer les amazones par exemple) grâce à GtkDragSource et GtkDropTarget. On peut tester avec GtkDragSource si l'on est bien positionné sur une amazone valide (exemple : ne pas pouvoir choisir une amazone de l'autre joueur). Lorsqu'on relâche l'amazone, on fait appel à GtkDropTarget qui renverra les coordonnées (x, y) sur la fenêtre, que l'on peut ensuite passer dans une fonction de conversion pour traduire (x, y) en une case du plateau de jeu, et ainsi vérifier si cela correspond à une case valide (= si le déplacement correspond à un coup valide).

2.1.5 Intégration et CLI

Le module d'intégration est le pilier central de l'application. Pour garantir un code propre et éviter d'éparpiller les données dans des variables globales, nous avons regroupé tout l'état de l'application dans une structure centrale nommée `Application`. Cette structure contient toutes les dépendances du programme. Elle gère trois aspects essentiels :

— **GameConfig :** Elle contient la configuration du jeu et les paramètres immuables (mode verbeux, port réseau ...) chargés au lancement ;
— **GameEngine* :** Un pointeur vers l'instance unique du moteur de jeu ;
— **Contrôle de flux :** Des drapeaux atomiques (ex : `is_running`



) permettant une terminaison propre et la libération des ressources via une procédure de nettoyage.

En parallèle, pour découpler la saisie utilisateur de l'exécution, nous utilisons une structure `UserCommand`. Elle normalise les entrées : une commande textuelle brute transformée en un type énuméré, accompagné d'arguments validés, facilitant le travail du Dispatcher.

2.2 Algorithmes utilisés

2.2.1 Réseaux

Bien que le projet ne nécessite pas d'algorithmes complexes en réseau, la robustesse du module repose sur trois algorithmes :

- Un algorithme de reconstruction TCP qui accumulera les données brutes dans un buffer persistant jusqu'à détecter un caractère de fin de message, puis extraira la commande complète pour traiter la fragmentation des messages ;
- Un algorithme de multiplexage qui utiliserait poll() pour surveiller simultanément l'activité du clavier, du serveur et des clients avec un temps d'attente nul afin d'éviter les opérations bloquantes ;
- Un algorithme de Routage sélectif qui parcourra la liste des clients et ne relaiera les messages qu'aux joueurs partageant le même "game id" que l'expéditeur pour éviter qu'un joueur ne reçoive les coups d'une autre partie en cours.

2.2.2 IA

MiniMax//Alpha-Beta Dû à la complexité du jeu, l'élagage standard de ces algorithmes n'est pas suffisant. Nous utiliserons alors une recherche sélective (Forward Pruning [3]), nous permettant d'élaguer à partir de chaque profondeur, ne gardant que les k meilleurs coups. Celle-ci se base sur l'évaluation de mobilité ($Coups_{Moi} - Coups_{Adv}$), qui est une évaluation très rapide bien que moins précise. Elle complétera les autres évaluations (Queen Distance & King Distance, Relative Territory Heuristic).

Monte Carlo Tree Search (MCTS) Cet algorithme construit l'arbre de jeu de manière asymétrique et itérative. Au lieu de tout calculer, il effectue des simulations de parties pour estimer le taux de victoire des coups. En parallèle, il utilise un algorithme d'évaluation afin de sélectionner les meilleurs coups à explorer, optimisant le parcours des sous-arbres possibles.

Parmi les algorithmes d'évaluation, nous retrouvons :

- **Upper Confidence bounds for Trees (UCT)** [5] : UCT arbitre dynamiquement entre l'exploration de coups peu visités et l'exploitation des coups prometteurs, permettant de gérer le grand facteur de branchement sans nécessiter d'évaluation experte à chaque nœud ;
- **Machine Learning (ML)** : Nous utiliserons un réseau de neurones (CNN) pour estimer la probabilité de victoire d'une position donnée. Ce modèle remplace la phase de simulation du MCTS : au lieu de simuler une partie, l'arbre interroge le réseau de neurones pour obtenir une évaluation directe de la feuille, gagnant ainsi du temps de calcul.



2.2.3 Moteur de jeu

Génération par Ray-Casting L'algorithme de génération des coups doit gérer une complexité supérieure à celle des échecs en raison de la nature composée d'un tour (Déplacement + Tir). Nous utilisons une approche de "Ray-Casting" imbriquée. Dans un premier temps, l'algorithme projette les déplacements possibles de l'amazone selon les 8 axes. Pour chaque case d'arrivée valide, une seconde passe de rayons est lancée pour déterminer les cibles des flèches.

Implémentation Technique Pour chaque direction, le moteur charge un masque représentant un rayon théorique maximal. L'algorithme applique d'abord une intersection logique (`Rayon&Plateau`) pour isoler les obstacles potentiels sur la trajectoire. Une instruction processeur de Bitscan identifie alors l'index du premier obstacle rencontré. Un second masque est ensuite généré pour éteindre toutes les cases situées au-delà de cet obstacle, ne laissant actifs que les mouvements valides. Une attention particulière est portée à la phase de tir. Puisque l'amazone tire après s'être déplacée, sa case de départ est libérée mais le plateau global n'est pas encore mis à jour. On applique un XOR temporaire sur le bitboard du plateau. Cette opération rend l'ancienne position libre, permettant à l'algorithme de Ray-Casting de valider un tir traversant la case que l'amazone vient de quitter.

2.2.4 CLI

Le shell interactif repose sur une boucle infinie de type REPL (Read-Eval-Print Loop) qui gère les interactions homme-machine jusqu'à l'arrêt du programme.

1. Saisie L'acquisition des commandes est déléguée à la bibliothèque `GNU Readline`. Elle capture l'entrée utilisateur sous forme de chaîne brute, tout en offrant les fonctionnalités d'édition de ligne et d'historique.

2. Tokenization La chaîne brute est transmise au tokenizer qui utilise la fonction standard `strtok`. Cet algorithme découpe la chaîne en segments ("tokens") et les encapsule dans une structure intermédiaire dédiée, nommée `UserCommand`. Cette structure normalise l'entrée en isolant le type de commande (enum) et ses arguments dans un tableau fixe, facilitant leur traitement ultérieur.

3. Exécution L'interpréteur utilise un tableau de correspondance. Chaque entrée de ce tableau associe un mot-clé (ex : "move") à un pointeur de fonction spécifique et sa documentation. La fonction `command_exec` parcourt cette table pour trouver une correspondance avec `args[0]`. Si un match est trouvé, la fonction du moteur de jeu associée est invoquée avec les arguments parsés.



Architecture

3.1 Modules

3.1.1 IA

Pour la partie IA, le but était de pouvoir avoir une architecture simple à tester, simple d'utilisation, mais aussi performante et facilement modulable, en permettant l'ajout d'une IA juste à partir d'une classe et de la modification d'une liste.

Nous avons donc l'architecture présente dans la figure 4 pour répondre à ces besoins. Les paramètres donnés aux IAs seront fournis directement par le GameEngine, qui les passera au sein d'un dictionnaire (map) lors de l'initialisation, en fonction des paramètres de jeu (comme la profondeur maximale, ...). La fonction d'évaluation // la fonction de sélection feront également partie de ces paramètres, et seront assignées via init_ai.

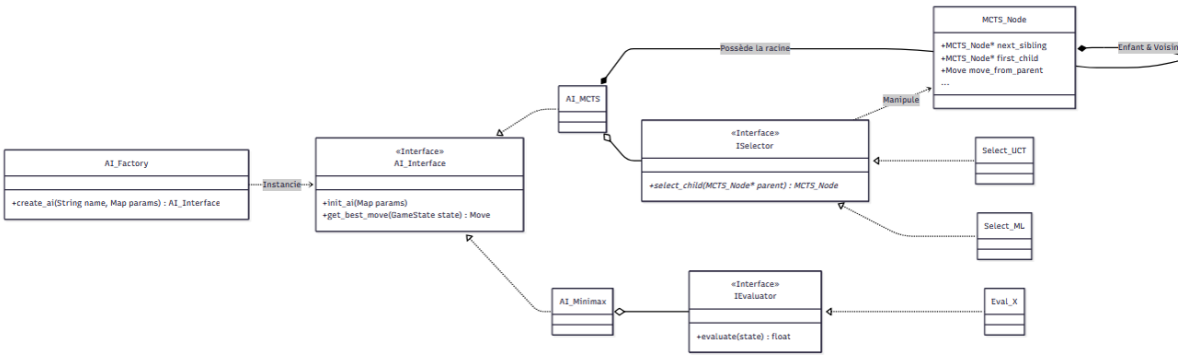


FIGURE 4 – IA - Diagramme UML

Comme précédemment cité (2.1.2), nous avons implémenté une structure d'arbre LCRS pour MCTS, et avons utilisé des interfaces pour le polymorphisme du code.

La dépendance entre le module IA et les autres modules ne se fera que sur le module de jeu principal, où GameEngine utilisera AI_Factory et gardera une référence à AI_Interface afin de pouvoir demander le meilleur coup.

3.1.2 Réseaux

On utilise plusieurs modules architecturaux en réseaux qui sont :

— Le premier est un pattern Façade qui n'est ni plus ni moins qu'une structure pivot contenant l'état global du réseau. Elle utilise le polymorphisme Ad-Hoc via un enum appelé "current mode" (exprimé par la cardinalité 0 ou 1 sur le diagramme). Cette structure permet au programme de décider dynamiquement d'utiliser le mode serveur et/ou client ou pas. Ce module à été choisit afin de centraliser la boucle

d'événements et d'éviter la duplication de code. En définitive, cela permet au programme de répondre a une contrainte majeure qui est de ne jamais se bloquer, auquel cas l'interface commencerait à geler, comme en utilisant la méthode poll(). Cela garantit donc que le réseau est traité avant de mettre à jour le plateau ou de redessiner l'écran.

— À noter que dans la structure globale du serveur, nous maintenons un socket d'écoute (listening socket) qui ne sert qu'à accepter les connexions, tandis que les structures ConnectedClient contiennent les sockets de communication dédiés.

— Finalement, on utilise dans le "NetworkManager" une interface Callback. Celle-ci sert au module Réseaux à ne pas dépendre des règles du jeu pour acheminer l'information au contrôleur, qui sera alors le seul à pouvoir traiter l'information reçue. De plus, cela permet d'éviter de créer une dépendance circulaire en devant appeler le jeu pour vérifier les coups. Le NetworkManager dit simplement "j'ai reçu tel packet et ce sera au contrôleur de décider quoi faire de l'événement que décrit ce packet".

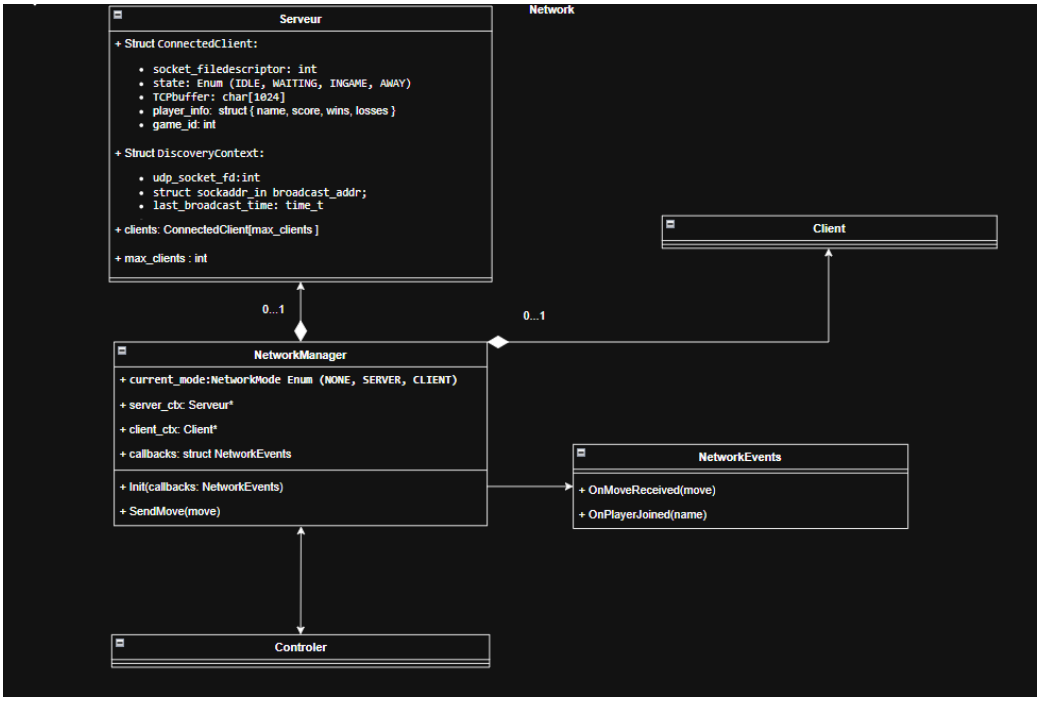


FIGURE 5 – Réseaux - Diagramme UML

3.2 Dépendances

Pour ce qui est des dépendances de l'architecture réseaux, on utilise essentiellement la bibliothèque native / système (POSIX) avec la Berkeley Sockets API, et la librairie readline comme demandée dans le sujet.

Pour l'IA, il y a une dépendance des Nodes du MCTS sur les Moves et avec les Bitboards.



Spécifications

4.1 Réseaux

4.1.1 Découverte de service

F38. Le client doit pouvoir voir automatiquement les serveurs de jeu actifs sur le réseau local via un broadcast, éliminant le besoin de connaître l'adresse IP cible à l'avance.

Prérequis : GUI (F28) et CLI (F15), Support du protocole UDP et configuration du socket avec l'option "SO BROADCAST".

Architecture découplée : Le serveur utilise une structure (DiscoveryContext) pour émettre périodiquement des paquets (contenant : Nom du serveur, Port TCP, Statut) sans bloquer le jeu. Le client écoute sur un port (ex : 8081).

Signatures des fonctions :

Serveur :

```
void broadcast_presence(int udp_socket, int tcp_port, const char* server_name);
```

Client :

```
server_list_t* client_scan_for_servers(int timeout_ms);
```

Validation : Lancer une instance serveur sur une machine A. Sur une machine B (sans configurer préalablement), lancer la commande server list et vérifier que l'IP et le nom de la machine A apparaissent immédiatement. Vérifier avec Wireshark que les paquets UDP sont bien envoyés à l'adresse de diffusion 255.255.255.255. Vérifier également que l'affichage GUI fonctionne bien, et que les commandes CLI sont bien existantes.

Transparence IA

Durant le projet, l'IA (Gemini 3.0) a été utilisée à la manière d'un moteur de recherche, et également à but informatif. Des exemples d'utilisations sont :

- Résumé d'informations contenues dans des papiers de recherche ;
- Explication de concepts clés et vulgarisation scientifique (exemple : explication calculs Queen Distance & King Distance, ...) ;
- Vérification de logique directe d'architecture ;
- Exemples réduits de codes possibles, par rapport à d'autres scénario (exemple : pseudo-code MCTS pour comprendre le fonctionnement de son architecture) ;
- Aide à la mise en page du rapport (certaines commandes Latex).



Références bibliographiques

[1] Louis Victor Allis. *Searching for Solutions in Games and Artificial Intelligence*. Ph.D. Thesis. 1994. URL : http://fragrieu.free.fr/SearchingForSolutions.pdf (visité le 21/01/2026).

[2] Patrick Hensgens. *A Knowledge-based Approach of the Game of Amazons*. Master's Thesis. 2001. URL : https://project.dke.maastrichtuniversity.nl/games/files/msc/Hensgens_thesis.pdf (visité le 20/01/2026).

[3] Qian Liang. *The evolution of Mulan : some studies in-game trees pruning and evaluation functions in the game of Amazons*. 2003. URL : https://sites.santafe.edu/~moore/QianThesis.pdf (visité le 21/01/2026).

[4] Timothy Furtak et al. *Generalized Amazons is PSPACE-complete*. 2005. URL : https://www.ijcai.org/Proceedings/05/Papers/1300.pdf (visité le 29/01/2026).

[5] Levente Kocsis et Csaba Szepesvari. *Bandit based Monte-Carlo Planning*. 2006. URL : http://web.archive.org/web/20070227171240/https://zaphod.aml.sztaki.hu/papers/ecml06.pdf (visité le 29/01/2026).

[6] John Tromp et Gunnar Farnebäck. *Combinatorics of Go*. 2007. URL : https://tromp.github.io/go/gostate.pdf (visité le 21/01/2026).

[7] Richard J Lorentz. *Amazons discover monte-carlo*. Springer. 2008. URL : https://link.springer.com/chapter/10.1007/978-3-540-87608-3_2 (visité le 29/01/2026).

[8] Jiaxing Song et Martin Müller. *An Enhanced Solver for The Game of Amazons*. 2014. URL : https://webdocs.cs.ualberta.ca/~mmueller/ps/2014/2014-TCIAIG-amazons_solver-preprint.pdf (visité le 21/01/2026).

[9] Guoqi Zhang et al. *Mastering the Game of Amazons Fast by Decoupling Network Learning*. 2021. DOI : 10.1109/IJCNN52387.2021.9534274. URL : https://ieeexplore.ieee.org/document/9534274 (visité le 29/01/2026).

[10] GeeksforGeeks. *Left Child Right Sibling Representation of Tree*. 2024. URL : https://www.geeksforgeeks.org/left-child-right-sibling-representation-tree/ (visité le 21/01/2026).

[11] boardgamegeek. *Amazones - Règle du jeu de société*. boardgamegeek. URL : https://boardgamegeek.com/image/1152391/amazons (visité le 20/01/2026).

[12] Filip Rachunek. *Amazones - 1 turn board state*. brainking. URL : https://i.brainking.com/rules/amazons/03.gif (visité le 20/01/2026).

[13] Filip Rachunek. *Amazones - Default board state*. brainking. URL : https://i.brainking.com/rules/amazons/02.gif (visité le 20/01/2026).